Trabalho de Laboratório — Ponteiros Avançados em C

Programação em C | Vetores, matrizes e estruturas | Atividade individual ou em grupo

Objetivo

Projetar e implementar um pequeno sistema de acompanhamento de alunos e notas, praticando a passagem de vetores e matrizes para funções, estruturas, ponteiros para estruturas e parâmetros de saída por endereço. Serão usados apenas caracteres isolados (tipo char), sem strings, ordenação ou alocação dinâmica.

O trabalho será realizado em duas etapas: Etapa 1 — em casa, elaboração e entrega somente do pseudocódigo; Etapa 2 — laboratório, implementação correspondente em C e apresentação.

Contexto

Uma turma contém até MAX_ALUNOS alunos e até MAX_DISC disciplinas. Cada aluno será identificado por um número inteiro de matrícula (campo matricula, tipo int), terá uma idade inteira e um turno representado por um caractere: M (manhã), T (tarde) ou N (noite). As notas inteiras serão armazenadas em uma matriz, com uma linha por aluno e uma coluna por disciplina. Não serão utilizados nomes, palavras ou vetores de caracteres.

Constantes e dimensões: MAX_ALUNOS = 50 (capacidade máxima da turma); MAX_DISC = 5 (capacidade máxima de disciplinas); n = número efetivo de alunos (1 a 50); m = número efetivo de disciplinas (1 a 5). Cada linha da matriz ocupa MAX_DISC posições, mesmo quando m é menor.

ETAPA 1 — Preparação em casa

Elaborar o algoritmo principal e os cinco subprogramas abaixo, somente em pseudocódigo. Indicar claramente os parâmetros que representam endereços. Não escrever código C nesta etapa.

1. lerDados

Recebe o vetor de alunos, a quantidade n, a matriz de notas e a quantidade m. Lê a matrícula numérica, a idade, o turno (M, T ou N) e m notas de cada aluno. As notas devem ser inteiras de 0 a 10. Para ler a matrícula, utilizar scanf("%d", ...); para o turno, scanf(" %c", ...).

2. calcularMedia

Recebe a matriz, n, m e o vetor de médias. Calcula a média aritmética das m notas de cada aluno e armazena no vetor de saída.

3. alterarIdade

Recebe o endereço de uma estrutura Aluno e uma nova idade. Modifica somente a idade do aluno indicado, sem devolver uma estrutura por retorno.

4. buscarMaiorMedia

Recebe o vetor de médias, n e os endereços de duas variáveis de saída: índice do aluno com maior média e valor da maior média. Em caso de empate, considera o primeiro aluno.

5. mostrarResultados

Recebe o vetor de alunos, o vetor de médias e n. Apresenta matrícula, idade, turno e média de cada aluno na ordem original, sem modificar os dados.

Algoritmo principal

- Ler n e m e verificar se estão dentro dos limites.
- Chamar lerDados e calcularMedia.
- Ler um índice de aluno válido (de 1 a n) e uma nova idade; chamar alterarIdade para esse aluno.
- Chamar buscarMaiorMedia para obter o índice e o valor da maior média.
- Chamar mostrarResultados e exibir o aluno com maior média.

Restrições da Etapa 1

- Usar subprogramas; o algoritmo principal apenas coordena as chamadas.
- Não utilizar ordenação, alocação dinâmica, ponteiros para ponteiros ou arquivos.
- Não repetir cálculos no algoritmo principal.
- Preservar a correspondência entre aluno, linha de notas e média pelo mesmo índice.
- Representar a diferença entre valor, endereço e alteração por referência.

🤖ETAPA 2 — Laboratório em C

Implementar o pseudocódigo em C99. Os protótipos devem aparecer antes de main(), e as definições das funções depois de main(). Usar preferencialmente colchetes [] para acessar vetores e matrizes. Reservar * e -> para os parâmetros por endereço e para estruturas apontadas.

```
#include <stdio.h>
#define MAX_ALUNOS 50
#define MAX_DISC 5

typedef struct {
    int matricula;       /* numero inteiro de identificacao */
    int idade;
    char turno;          /* M, T ou N */
} Aluno;

void lerDados(Aluno alunos[], int n, int notas[][MAX_DISC], int m);
void calcularMedia(int notas[][MAX_DISC], int n, int m, float medias[]);
void alterarIdade(Aluno *aluno, int novaIdade);
void buscarMaiorMedia(float medias[], int n, int *indice, float *maior);
void mostrarResultados(Aluno alunos[], float medias[], int n);

int main(void) {
    /* declaracoes, leituras e chamadas */
    return 0;
}
/* definicoes das cinco funcoes apos main */
```

Observações: notas[][MAX_DISC] descreve linhas com MAX_DISC colunas; m indica quantas colunas são usadas. Para ler matrícula, utilizar scanf("%d", &alunos[i].matricula); para ler turno, utilizar scanf(" %c", &alunos[i].turno). Para exibir matrícula e turno, usar respectivamente %d e %c. A função alterarIdade deve empregar aluno->idade = novaIdade;.

🤓Cenários de referência

Cenário 1 — n=3, m=2. Matrícula 101 (20 anos, turno M): 8 e 6; matrícula 102 (21 anos, turno T): 10 e 8; matrícula 103 (19 anos, turno N): 5 e 7. Médias: 7,0; 9,0; 6,0. Alterar a idade do aluno de matrícula 101 para 22. Maior média: matrícula 102, 9,0. A ordem de exibição continua 101, 102, 103.

Cenário 2 — n=2, m=3. Matrícula 201 (18 anos, turno M): 7, 7, 7; matrícula 202 (20 anos, turno T): 8, 6, 7. Ambas as médias são 7,0. O resultado de buscarMaiorMedia deve indicar a matrícula 201, por ser o primeiro aluno.

? Questões adicionais — apresentação no laboratório

O professor escolherá uma pequena alteração para ser implementada ao vivo, em aproximadamente cinco minutos. As questões abaixo não são exigidas na entrega inicial.

- Questão A — contarAprovados: criar int contarAprovados(float medias[], int n); para contar quantos alunos têm média maior ou igual a 6,0.
- Questão B — mediaDisciplina: criar void mediaDisciplina(int notas[][MAX_DISC], int n, int m, int disciplina, float *media); para calcular a média de uma coluna indicada por índice válido de 0 a m-1.
- Questão C — alterarNota: criar void alterarNota(int notas[][MAX_DISC], int aluno, int disciplina, int novaNota); para modificar uma nota inteira de 0 a 10 na posição indicada por índices válidos (aluno de 0 a n-1; disciplina de 0 a m-1). Após a alteração, chamar novamente calcularMedia() e mostrarResultados() para observar a atualização das médias.

? Entrega e apresentação

Etapa 1: entregar somente o pseudocódigo do algoritmo principal e das cinco funções obrigatórias. Etapa 2: apresentar o programa C em execução, com as funções completas e testes dos cenários. Durante a apresentação, explicar a passagem da matriz, a diferença entre . e ->, a leitura de caracteres isolados com %c e o uso de parâmetros de saída por endereço.

Critérios de avaliação: decomposição correta em funções; correspondência entre pseudocódigo e C; uso correto de parâmetros e endereços; resultados e validação básica; clareza da explicação oral.